



Formalising Knowledge

Formal Concept Analysis & Association Rules

Nikola Simidjievski
Télécom Paris, Institut Polytechnique de Paris
nikola.simidjievski@telecom-paris.fr
9 octobre 2026

The Knowledge Discovery Process

An interactive and iterative process guided by an analyst and knowledge of the domain.

Data Preparation

- ➜ Data acquisition
- ↓ Measure, crawl, scrape,...
- ➜ Raw data
- ↓ Selection and preparation
- ↓ Transformation : cleaning and formatting
- ➜ **Prepared data**

Mining & Knowledge

- ➜ Prepared data
- ↓ Data mining (Numerical & symbolic methods)
- ➜ Outputs
- ↓ Interpretation and evaluation
- ↓ Knowledge representation
- ↓ Reasoning w/ knowledge representations
- ➜ **Knowledge based systems**

The Knowledge Discovery Process

- Data comes in large volumes, we need to sift through, and extract useful/significant knowledge.
- Tasks of the "data $\rightarrow$ knowledge" process :
  - **Representing :** Concepts and relations as knowledge representations
  - **Modeling :** (hierarchies of) Concepts and relations
  - **Mining :** (closed) frequent ...) pattern mining (itemsets, sequences, graphs,...)
  - **Reasoning :** Case-based reasoning and classification
- Many domains of applications :
  - Scientific data (medicine, biology, chemistry...)
  - Social data (documenting human behaviors ...)
  - Housekeeping data (large software repositories ...)

Agenda

- How can we formalize, represent and model knowledge ?
  - **Formal Concept Analysis**
- How do we find (interesting/meaningful) patterns and reason ?
  - **Association Rules**

We are given some data...

	y1	y2	y3	y4	y5
x1	×				×
x2	×	×	×	×	×
x3	×		×		
x4			×		×
x5		×	×	×	×
x6	×	×	×		×
x7	×		×	×	×
x8	×	×	×		
x9	×			×	
x10		×	×		×

- Here is some data, that a doctor collected from his patients
- A set of patients X_i
- A set of symptoms Y_j
- The relation states that some patients had some symptoms
- This is a very vanilla example
  - Data in reality is noisy
  - Data is usually much much larger (thousands of rows and columns)

We are given some data...

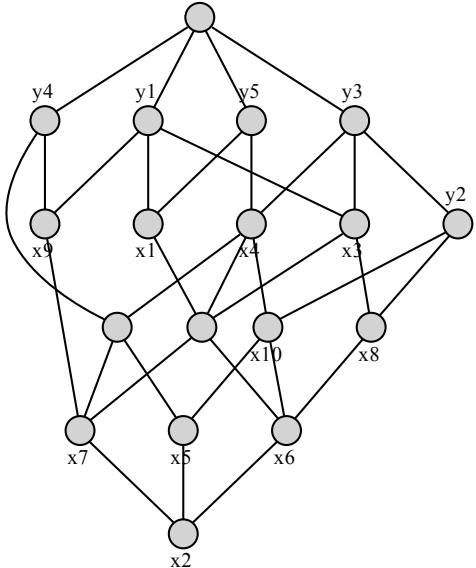
	y1	y2	y3	y4	y5
x1	×				×
x2	×	×	×	×	×
x3	×		×		
x4			×		×
x5		×	×	×	×
x6	×	×	×		×
x7	×		×	×	×
x8	×	×	×		
x9	×			×	
x10		×	×		×

- $\{y2, y3, y5\}$ is a set of frequency $4/10 = 0.4$.
- $\{y3, y5\}$ has $6/10 = 0.6$ as frequency.
- $y3 \wedge y5 \longrightarrow y2$ is an association with a confidence of $4/6 = 0.66$:
  - If a patient has symptom y3 and y5, they also have y2 with probability 0.66.

We are given some data...

	y1	y2	y3	y4	y5
x1	×				×
x2	×	×	×	×	×
x3	×		×		
x4			×		×
x5		×	×	×	×
x6	×	×	×		×
x7	×		×	×	×
x8	×	×	×		
x9	×			×	
x10		×	×		×

What else can we learn ?



Formal Concept Analysis

Formal Concept Analysis

- **What ?**
  - A framework for formalization (and reasoning) of knowledge structures present in data
- **How ?**
  - A concept hierarchy from collection of objects (samples) and properties (variables) given as a context (data)
- **Why ?**
  - Data Mining & Knowledge Discovery
    - Feature Extraction
    - Association Rules Induction
  - Machine learning
    - Rule-based classifiers (Decision Trees)
    - Feature selection & engineering

Inputs & Outputs of FCA

Input :

- ➜ Data (context) as table
  - objects (rows)
  - attributes/properties (columns)

Output :

- ➜ Concept Lattice :
  - *hierarchy of formal concepts* (nodes of a Hasse diagram)
  - (partially ordered) sub-concepts/super-concepts, as it's a hierarchy
  - eg. "patients with symptoms y_3 and y_5 "
- ➜ Attribute implications (the Duquenne-Guigues basis)
  - set of dependencies in the data ("implicit rules")
  - eg. "every patient with symptom y_2 also has a symptom y_3"
  - eg. "every even number divisible by 3, is also divisible by 6"

The Formal Context (Input Data)

The starting point is **formal context** $\mathbb{K}$, represented as a triplet $\mathbb{K} = (X, Y, I)$:

- X is a set of **objects**.
- Y is a set of **attributes**.
- $I \subseteq X \times Y$ is a binary relation where $(x, y) \in I$ means "object x has attribute y".

Or given as a table :

attributes

I	y_1	y_2	y_3	y_4
x_1	×	×	×	×
x_2	×		×	×
x_3		×	×	×
x_4		×	×	×
x_5	×			

objects

Eg : The object x_1 has attributes y_1, y_2, y_3, y_4.

Formal Concepts

A **formal concept** is a pair (A, B) where $A \subseteq X$ and $B \subseteq Y$ such that :

$$A^{\uparrow} = B \quad \text{and} \quad B^{\downarrow} = A$$

- A is called the **extent** of the concept (all its objects).
- B is called the **intent** of the concept (all its attributes).

Interpretation

A formal concept is a perfect pairing :

- The extent consists of **all** objects that share**all** the attributes in the intent
- The intent consists of **all** attributes shared by **all** the objects in the extent.

Example : $(\{x_1, x_3, x_4\}, \{y_2, y_3, y_4\})$ is a formal concept.

Derivation Operators (or, how do we find concepts ?)

From a formal context, we define a pair of derivation operators :

- For a set of objects $A \subseteq X$, $A^{\uparrow}$ is the set of attributes common to **all** objects in A
  $\Rightarrow A^{\uparrow} = \{y \in Y \mid \forall x \in A : (x, y) \in I\}$
- For a set of attributes $B \subseteq Y$, $B^{\downarrow}$ is the set of objects that have **all** attributes in B
  $\Rightarrow B^{\downarrow} = \{x \in X \mid \forall y \in B : (x, y) \in I\}$

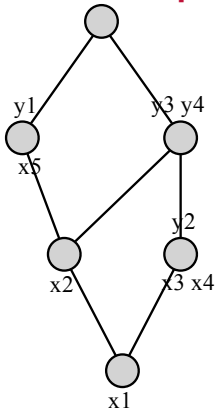
Example

Using the previous table :

- Start with $\{y_2\} : \{y_2\}^{\downarrow} = \{x_1, x_3, x_4\}$.
- Return to attributes : $\{x_1, x_3, x_4\}^{\uparrow} = \{y_2, y_3, y_4\}$.

These operators form a mathematical structure called a **Galois connection**.

The Concept Lattice

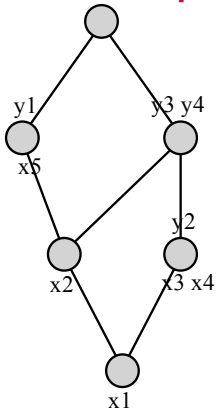


Formal concepts are ordered by a subconcept-superconcept relation :

$$(A_1, B_1) \preceq (A_2, B_2) \iff A_1 \subseteq A_2 \text{ (equivalent to } B_2 \subseteq B_1)$$

- ➜ A concept is more specific (a subconcept) if it has fewer objects but more attributes
- ➜ A concept is more general (a superconcept) if it captures more objects, but less attributes

The Concept Lattice



Formal concepts are ordered by a subconcept-superconcept relation :

$$(A_1, B_1) \preceq (A_2, B_2) \iff A_1 \subseteq A_2 \text{ (equivalent to } B_2 \subseteq B_1)$$

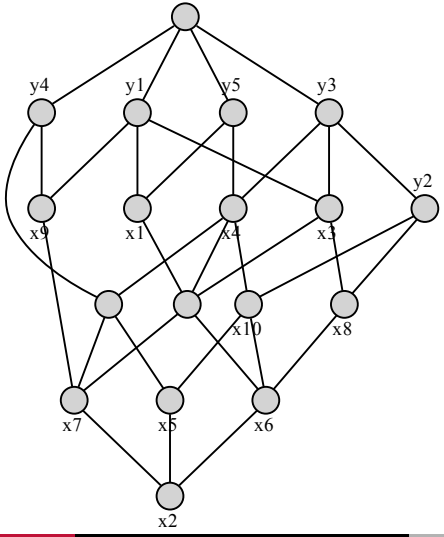
- ➜ A concept is more specific (a subconcept) if it has fewer objects but more attributes
- ➜ A concept is more general (a superconcept) if it captures more objects, but less attributes

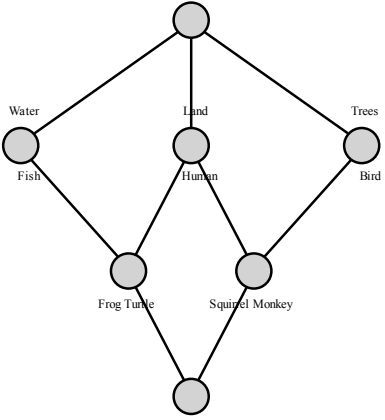
Concept Lattice

The poset $(\mathbb{C}(\mathbb{K}), \preceq)$ of all formal concepts of the context $\mathbb{K} = (X, Y, I)$ is a *complete lattice* called **concept lattice.**

This lattice is often visualized as a Hasse diagram.

The lattice


y4
y1
y5
y3
y2
x9
x1
x4
x3
x10
x8
x7
x5
x6
x2


Water
Land
Trees
Fish
Human
Bird
Frog Turtle
Squirrel Monkey

The Galois Connection

The pair of operators $(\cdot^{\uparrow}, \cdot^{\downarrow})$ forms a **Galois Connection**. This is a fundamental structure in FCA, defined by four properties for any object sets A_1, A_2 and attribute sets B_1, B_2 :

1. Antitone Property

- $A_1 \subseteq A_2 \implies A_2^{\uparrow} \subseteq A_1^{\uparrow}$
- $B_1 \subseteq B_2 \implies B_2^{\downarrow} \subseteq B_1^{\downarrow}$

Adding elements to the input set can only shrink the output set

2. Closure Property

- $A_1 \subseteq (A_1^{\uparrow})^{\downarrow}$
- $B_1 \subseteq (B_1^{\downarrow})^{\uparrow}$

Applying both operators returns a superset of the original set.

From Galois Connection to Closure

By composing the Galois connection operators, we create **closure operators**.
A mapping $T : 2^S \to 2^S$ is a closure operator if it is :

- **Extensive :** $A \subseteq T(A)$
- **Monotone :** $A \subseteq B \implies T(A) \subseteq T(B)$
- **Idempotent :** $T(T(A)) = T(A)$

From Galois Connection to Closure

By composing the Galois connection operators, we create **closure operators**.
A mapping $T : 2^S \to 2^S$ is a closure operator if it is :

- **Extensive :** $A \subseteq T(A)$
- **Monotone :** $A \subseteq B \implies T(A) \subseteq T(B)$
- **Idempotent :** $T(T(A)) = T(A)$

The Closure Operators in FCA

The composition $(\cdot)^{\uparrow\downarrow}$ is a closure operator :

- The operator $(\cdot)^{\uparrow\downarrow}$ on **object sets** A finds the full **extent** of a concept.
- The operator $(\cdot)^{\downarrow\uparrow}$ on **attribute sets** B finds the full **intent** of a concept.

The formal concepts are precisely the "closed" sets under these operators.

Lattice Operations : Meet and Join

For any collection of concepts $\{(X_t, Y_t)\}_{t \in T}$, their greatest lower bound (**infimum** or **meet**) and least upper bound (**supremum** or **join**) also exist within the concept lattice :

- Infimum/Meet : Intersection of extents. Its intent is the closure of the union of the original intents.

$$\bigwedge_{t \in T} (X_t, Y_t) = \left(\bigcap_{t \in T} X_t, \left(\bigcup_{t \in T} Y_t \right)^{\downarrow\uparrow} \right)$$

- Supremum/Join : Intersection of intents. Its extent is the closure of the union of the original extents

$$\bigvee_{t \in T} (X_t, Y_t) = \left(\left(\bigcup_{t \in T} X_t \right)^{\uparrow\downarrow}, \bigcap_{t \in T} Y_t \right)$$

*Infimum/Supremum concepts are **formal concepts***

Computing the Lattice

Computing the Lattice : NextClosure Algorithm

The number of formal concepts can be very large, so we need efficient algorithms to find them.
NextClosure is a classic algorithm for this task :

- **Goal :** One by one, generate all formal concept intents (in lexicographical order)
- Using the intents, compute the extents to get the full concepts.
- Ideal for knowledge exploration on small contexts. Small memory footprint.
- Complexity $O(|X||Y|^2|C|)$ (for the standard attribute-based version, where C is the set of concepts)

NextClosure : A Step-by-Step Example

Let's trace the algorithm on the context from our example.

I	y_1	y_2	y_3	y_4
x_1	×	×	×	×
x_2	×		×	×
x_3		×	×	×
x_4		×	×	×
x_5	×			

- ➜ The algorithm generates closed intents in lexicographical order by finding the next valid intent from the current one. (Set $y_1 < y_2 < y_3 < y_4$)
- ➜ Starting from the highest-ordered attribute (y_4), making the smallest change possible.
  - ie. try attributes from y_4 downwards ; accept only if closure adds no new earlier attribute.
- ➜ **Full walkthrough in the appendix**

I	y_1	y_2	y_3	y_4
x_1	×	×	×	×
x_2	×		×	×
x_3		×	×	×
x_4		×	×	×
x_5	×			

1. Start with $\emptyset$: no attributes are required, so all objects satisfy it. *Extent :* $\{x_1, x_2, x_3, x_4, x_5\}$, *Intent :* $\emptyset$.
2. Next intent $\{y_3, y_4\}$: objects having y_4 also have y_3. *Extent :* $\{x_1, x_2, x_3, x_4\}$, *Intent :* $\{y_3, y_4\}$.
3. Add y_2 : objects with y_2 are $\{x_1, x_3, x_4\}$, which also share y_3, y_4. *Extent :* $\{x_1, x_3, x_4\}$, *Intent :* $\{y_2, y_3, y_4\}$.
4. Move to y_1 : objects with y_1 are $\{x_1, x_2, x_5\}$, with no additional shared attribute. *Extent :* $\{x_1, x_2, x_5\}$, *Intent :* $\{y_1\}$.
5. Add y_3 to y_1 : objects with y_1, y_3 are $\{x_1, x_2\}$, and both also have y_4. *Extent :* $\{x_1, x_2\}$, *Intent :* $\{y_1, y_3, y_4\}$.
6. Finally add y_2 : only x_1 has y_1, y_2, and x_1 has all four attributes. *Extent :* $\{x_1\}$, *Intent :* $\{y_1, y_2, y_3, y_4\}$.

Final List of Concepts :

$$C_1 = (\{x_1, x_2, x_3, x_4, x_5\}, \emptyset)\,;\; C_2 = (\{x_1, x_2, x_3, x_4\}, \{y_3, y_4\})\,;C_3 = (\{x_1, x_3, x_4\}, \{y_2, y_3, y_4\})\,;$$
$$C_4 = (\{x_1, x_2, x_5\}, \{y_1\})\,;C_5 = (\{x_1, x_2\}, \{y_1, y_3, y_4\})\,;\; C_6 = (\{x_1\}, \{y_1, y_2, y_3, y_4\})$$

Some other alternatives

- **Close-by-One (CbO) (Kuznetsov, 1993) :**
  - Canonicity testing to avoid generating duplicate concepts. Complexity $O(|X|^2|Y||C|$. Several extensions and improvements.
- **Titanic (Stumme et al., 2002) :**
  - Level-wise algorithm using support functions for efficient computation of **iceberg** lattices :).
  - Practical performance depends on context size, density, implementation, and whether the diagram edges are also constructed
- **Bordat (Bordat, 1986) :**
  - Spanning Tree-based storage with $O(|X||Y|^2|C|)$ complexity, performs best on large contexts of average density.
- **Chein (Chein, 1969) :**
  - Generates concepts by intersecting intents of previously generated concepts in layers with $O(|X|^3|Y||C|)$ complexity.
- **Lindig (Lindig, 2000) :**
  - Bottom-up approach generating upper neighbors for each concept with $O(|X|^2|Y||C|)$ complexity.

Context pre-processing (in hindsight)

Context clarification and Reduction

A good pre-processing step for efficiency

- ➜ It can be redundant if it contains identical rows or columns.
- ➜ Clarifying/reducing a context results in isomorphic (structurally identical) concept lattice.

Clarified Context

A context is **clarified** if it contains no identical objects and no identical attributes.

Reduced Context

An object or attribute is **reducible** if its information can be derived from others. A **reduced context** has no reducible objects or attributes.

A reduced context is always clarified, but a clarified context is not necessarily reduced.

In other words

Clarified Context

- **Goal :** Remove simple redundancies.
- **Method :** Remove identical rows (objects with the same attributes) and identical columns (attributes of the same objects).
- **Result :** A smaller context with a structurally identical (isomorphic) lattice.

Reduced Context

- **Goal :** Remove structural redundancies.
- **Method :** Remove objects or attributes whose information is derivable from others.
- **Result :** A minimal context that can still generate the full lattice.

Example : Clarifying the Context

We start with our original context and look for identical rows or columns.

Original Context

I	y_1	y_2	y_3	y_4
x_1	×	×	×	×
x_2	×		×	×
x_3		×	×	×
x_4		×	×	×
x_5	×			

- Objects x_3 and x_4 have the same attributes.
- Attributes y_3 and y_4 apply to the same objects.

Clarified Context

I	y_1	y_2	y_3
x_1	×	×	×
x_2	×		×
x_3		×	×
x_5	×		

- We remove the duplicates (x_4 and y_4) to get the clarified context.
- The conceptual structure (the lattice) remains unchanged.

Implications

Attribute Implications

The second major output of FCA is a set of **attribute implications** :

Given $K = (X, Y, I), Z_1, Z_2 \subseteq Y$

- An expression of the form Z_1 (premise) $\Rightarrow Z_2$ (conclusion), where $Z_1, Z_2 \subseteq Y$,
- It means : "Any object that has all attributes in Z_1 also has all attributes in Z_2."

Attribute Implications

Validity

An implication $Z_1 \Rightarrow Z_2$ is **true** in a formal context (X, Y, I) if every **object** that has all attributes from Z_1 also has all attributes from Z_2 :

$$\text{Formally : } Z_1^{\downarrow} \subseteq Z_2^{\downarrow}$$

I	y_1	y_2	y_3	y_4
x_1	×	×	×	×
x_2	×		×	×
x_3		×	×	×
x_4		×	×	×
x_5	×			

- $\{y_2\} \Rightarrow \{y_3\}$ is **valid**
- $\{y_1, y_2\} \Rightarrow \{y_4\}$ is **valid**
- $\{y_1\} \Rightarrow \{y_3\}$ is **not valid**

Reasoning with Implications

We can derive new implications from a given set using a sound and complete system of inference rules :

Armstrong's Axioms

- **Reflexivity :** If $Z_2 \subseteq Z_1$, then $Z_1 \Rightarrow Z_2$
  (Trivial : If a car has ABS and an airbag, it has an airbag).
- **Augmentation :** If $Z_1 \Rightarrow Z_2$, then $Z_1 \cup Z_3 \Rightarrow Z_2 \cup Z_3$
  (If automatic implies AC, then automatic and sunroof implies AC and sunroof).
- **Transitivity :** If $Z_1 \Rightarrow Z_2$ and $Z_2 \Rightarrow Z_3$, then $Z_1 \Rightarrow Z_3$
  (If automatic implies AC, and AC implies a higher price, then automatic implies a higher price).

The goal is to find a small, non-redundant set of implications, called a **basis**, from which all other valid implications can be derived.

Non-Redundant Basis

A set of implications T is a **non-redundant basis** of a formal context if :

1. It is **complete** : all implications true in the context can be derived from T.
2. It is **non-redundant** : no implication in T can be derived from the others in T.

The Duquenne-Guigues Basis

- ➜ A canonical, non-redundant basis can be computed from the **pseudo-intents** of the formal context.
- ➜
- ➜ **Pseudo-intents** are non-closed sets of attributes used as premises in the DG basis. Each premise implies the additional attributes in its closure. This basis has the minimum number of implications needed to derive all valid implications.

Conclusion

Extensions of FCA

Data is messy and complex...

- Many-values context :
  (X, Y, W, I); $I \subseteq X \times Y \times W$, where W is a set of values
- Non-binary context :
  Scaling, pattern structures
- Many complex patterns :
  Icebergs, stability, ...
- n-dimensional context :
  Triadic and polyadic concept analysis
- Relations between objects themselves :
  Relational concept analysis

Recap

- **Idea :** A mathematical framework to derive a concept hierarchy from data.
- **Input :** A **formal context** $\mathbb{K} = (X, Y, I)$, a table where X are objects, Y are attributes, and I is the relation between them.
- A **formal concept** is a pair (**extent, intent**) representing a maximal grouping of objects sharing common attributes.
- **Outputs :**
  - **Concept Lattice :** A complete lattice, a poset $(\mathbb{C}, \preceq)$ of formal concepts, embodying the subconcept/superconcept hierarchy ;
    Computed using algorithms like **NextClosure**, *after* (optional) context clarification/reduction
  - **Attribute Implications :** A minimal, non-redundant set of rules that are perfectly true in the context.

Further Resources

- An extensive resource on FCA `https://upriss.github.io/fca/fca.html`
- R. Belohlavek "Introduction to Formal Concept Analysis" (you can get it here : `http://belohlavek.inf.upol.cz/vyuka/IntroFCA.pdf`
- Claudio Carpineto and Giovanni Romano, Concept Data Analysis : Theory and Applications,2004.
- Nathalie Caspard, Bruno Leclerc and Bernard Monjardet, Finite Ordered Sets : Concepts, Results and Uses, 2012.
- Bernhard Ganter and Rudolf Wille, Formal Concept Analysis, 1999.
- Franz Baader, Bernhard Ganter, Baris Sertkaya, Ulrike Sattler, Completing Description Logic Knowledge Bases Using Formal Concept Analysis. IJCAI 2007 : 230-235
- Oded Maimon, Lior Rokach (Eds.), The Data Mining and Knowledge Discovery Handbook, 2005.

Question ? (thus far)

So ...

Recall, the tasks of the "data → knowledge" process :

- **Representing :** Concepts and relations as knowledge representations
- **Modeling :** (hierarchies of) Concepts and relations
- **Mining**
  - Data mining
  - Frequent pattern mining
  - Knowledge discovery
  - ...
- **Reasoning**

Association Rules

Example

The Market Model

- We have a large set of **items**.
- We have a large set of **baskets** (or transactions).

ID	Items
T1	novel, newspaper, music
T2	novel, film, comics, contemporary music
T3	newspaper, film, comics, classical music
T4	novel, newspaper, film, comics
T5	novel, newspaper, film, classical music

Goal

Identify items that are bought together by sufficiently many customers. "People who bought {item A, item B} also tend to buy {item C}."

1. Process sales data collected with barcode scanners to find dependencies among items.
2. Design strategic product placement and promotions.
3. ...
4. Profit :)

Examples of rules

- ➜ $\{\text{film}\} \Rightarrow \{\text{comics}\}$
- ➜ $\{\text{newspaper, novel}\} \Rightarrow \{\text{film, classical music}\}$
- ➜ $\{\text{comics, novel}\} \Rightarrow \{\text{newspaper}\}$

$X \Rightarrow Y$: If all attributes in X are present in a set, the attributes in Y *tend* to be present too.
This refers to *co-occurrence*, **NOT** *causality*

Modern Application : E-commerce

Amazon's "Frequently bought together" feature is a direct application of this approach.

But not only retail...

The concepts of "items" and "baskets"/"collections" (or more generally "itemsets") can represent many different kinds of relationships :

- ➜ Items / Itemset $\rightarrow$ **Words** / **Documents**
  - Finding words that frequently appear together in the same documents (eg. plagiarism detectors)
- ➜ Items / Itemset $\rightarrow$ **Basepairs** / **Genes**
  - Finding sets of genetic markers that co-occur in certain genes.
- ➜ Items / Itemset $\rightarrow$ **Drugs** / **Patients**
  - Finding combinations of medications prescribed to the same patients.
  - “Identify combinations of drugs associated with reported adverse effects.

Definitions

- **Itemset :** A collection of one or more items.
  - Example : $X = \{\text{novel, film, comics}\}$

Definitions

- **Itemset :** A collection of one or more items.
  - Example : $X = \{\text{novel, film, comics}\}$
- **Support Count** $\sigma(\cdot)$ **:** The number of transactions containing a specific itemset.
- **Support** $s(\cdot)$ **:** The ratio of total transactions that contain the itemset ; I.e. itemset's "popularity".

$$s(X) = \frac{\sigma(X)}{\text{Total Number of Transactions}}$$

Definitions

- **Itemset :** A collection of one or more items.
  - Example : $X = \{\text{novel, film, comics}\}$
- **Support Count** $\sigma(\cdot)$ **:** The number of transactions containing a specific itemset.
- **Support** $s(\cdot)$ **:** The ratio of total transactions that contain the itemset ; I.e. itemset's "popularity".

$$s(X) = \frac{\sigma(X)}{\text{Total Number of Transactions}}$$

Example Calculation

Given 5 transactions, if the itemset $\{\text{newspaper, novel, film}\}$ appears in 2 of them (T4 and T5) :

- Support count : $\sigma(\{\text{newspaper, novel, film}\}) = 2$
- Support : $s(\{\text{newspaper, novel, film}\}) = 2/5 = 0.4$

Definitions

- **Itemset :** A collection of one or more items.
  - Example : $X = \{\text{novel, film, comics}\}$
- **Support Count** $\sigma(\cdot)$ **:** The number of transactions containing a specific itemset.
- **Support** $s(\cdot)$ **:** The ratio of total transactions that contain the itemset ; I.e. itemset's "popularity".

$$s(X) = \frac{\sigma(X)}{\text{Total Number of Transactions}}$$

Example Calculation

Given 5 transactions, if the itemset $\{\text{newspaper, novel, film}\}$ appears in 2 of them (T4 and T5) :

- Support count : $\sigma(\{\text{newspaper, novel, film}\}) = 2$
- Support : $s(\{\text{newspaper, novel, film}\}) = 2/5 = 0.4$

- **Frequent Itemset :** An itemset whose support is greater than or equal to a user-defined minimum support threshold minSup : $s(X) \geq \text{minSup}$.

Association Rule

An association rule is an expression of the form $X \Rightarrow Y$, where X and Y are non-empty itemsets.

Support

The support of a itemset containing both X and Y

$$s(X \Rightarrow Y) = \frac{\sigma(X \cup Y)}{|T|}$$

Confidence

Conditional probability of Y, given X

$$c(X \Rightarrow Y) = \frac{\sigma(X, Y)}{\sigma(X)}$$

Example : {newspaper, film} $\Rightarrow$ {comics}

$$s = \frac{\sigma(\{\text{newspaper, film, comics}\})}{|T|} = \frac{2}{5} = 0.4$$

$$c = \frac{\sigma(\{\text{newspaper, film, comics}\})}{\sigma(\{\text{newspaper, film}\})} = \frac{2}{3} \approx 0.67$$

Fantastic rules and where to find them...

Rule Mining

1. List all possible association rules (combinations)
2. Compute S and C of each rule
3. Prune rules that with $S < minSup$ or $C < minConf$ (user defined thresholds)

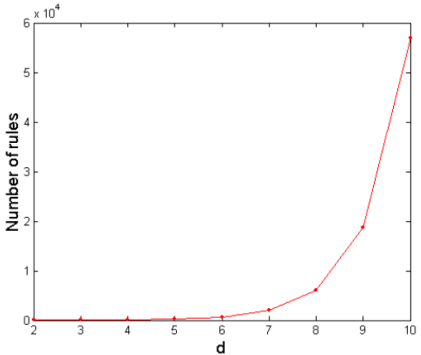
Rule Mining

1. List all possible association rules (combinations)
2. Compute S and C of each rule
3. Prune rules that with $S < minSup$ or $C < minConf$ (user defined thresholds)

The Problem

A brute-force approach is computationally expensive. For d items, there are 2^d possible itemsets and an even larger number of possible rules. We can't check them all.

$$R = \sum_{i}^{d-1} C_d^i \left(\sum_{j}^{d-i} C_{d-i}^j \right)$$



Rule Mining

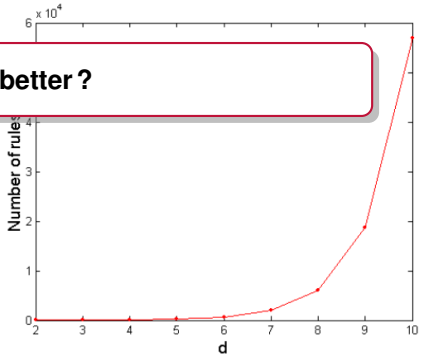
1. List all possible association rules (combinations)
2. Compute S and C of each rule
3. Prune rules that with $S < minSup$ or $C < minConf$ (user defined thresholds)

The Problem

A brute-force approach is very expensive. For d items, there are 2^d possible itemsets and an even larger number of possible rules. We can't check them all.

$$R = \sum_{i}^{d-1} C_d^i \left(\sum_{j}^{d-i} C_{d-i}^j \right)$$

Can we do better?



Some practical metrics

- **Most often** association rule algorithms are designed for datasets that are too large to fit into main memory. The data is typically stored on disk.
- Reading from disk is the **slowest** part of the mining process.

A Pass

A **pass** is one complete scan of the entire database from disk, from start to finish.

Measuring Cost

The efficiency of rule mining algorithms can be measured with **number of passes** required. The fewer passes an algorithm makes, the faster it will generally be.

The A Priori Principle

The A Priori Principle

If an itemset is frequent, then all of its subsets must also be frequent.

The A Priori Principle

The A Priori Principle

If an itemset is frequent, then all of its subsets must also be frequent.

The Contrapositive (Used for Pruning)

If an itemset is infrequent, then all of its supersets must also be infrequent.

The A Priori Principle

The A Priori Principle

If an itemset is frequent, then all of its subsets must also be frequent.

The Contrapositive (Used for Pruning)

If an itemset is infrequent, then all of its supersets must also be infrequent.

- This works because support is an **anti-monotone** property :

$$\forall X, Y : (X \subseteq Y) \Rightarrow s(X) \geq s(Y)$$

- An itemset cannot appear more often than any of its individual parts.

The A Priori Algorithm (Agrawal and Srikant, 1994)

A Level-by-Level Approach

The algorithm iteratively finds frequent itemsets of increasing size ($k = 1, 2, 3, ...$).

- **Pass 1 (L_1) :** Scan the database, count support of individual items. Keep only the frequent ones.

The A Priori Algorithm (Agrawal and Srikant, 1994)

A Level-by-Level Approach

The algorithm iteratively finds frequent itemsets of increasing size ($k = 1, 2, 3, ...$).

- **Pass 1 (L_1) :** Scan the database, count support of individual items. Keep only the frequent ones.
- **Pass 2 (L_2) :**
  - **Join Step :** Form candidate 2-itemsets (C_2) only from the frequent items in L_1.
  - **Prune Step :** Scan the database to count support of candidates in C_2. Keep the frequent ones.

The A Priori Algorithm (Agrawal and Srikant, 1994)

A Level-by-Level Approach

The algorithm iteratively finds frequent itemsets of increasing size ($k = 1, 2, 3, ...$).

- **Pass 1 (L_1) :** Scan the database, count support of individual items. Keep only the frequent ones.
- **Pass 2 (L_2) :**
  - **Join Step :** Form candidate 2-itemsets (C_2) only from the frequent items in L_1.
  - **Prune Step :** Scan the database to count support of candidates in C_2. Keep the frequent ones.
- **Pass k (L_k) :** Repeat the 'join and prune' process. Join frequent sets of size $k-1$ to create candidate sets of size k. Prune candidates containing an infrequent subset. Scan the database and keep those meeting minSup
- The algorithm stops when no new frequent itemsets can be found.

Visualizing A Priori Pruning

If we find that an itemset is infrequent, we can prune its entire branch of supersets.

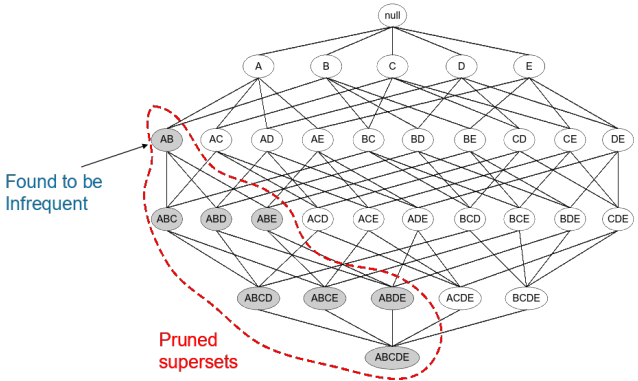


Figure – If {A,B} is found to be infrequent, all supersets like {A,B,C} and {A,B,D} are pruned without being counted.

Apriori : A tiny example

ID	Items
T1	novel, newspaper, music
T2	novel, film, comics, contemporary music
T3	newspaper, film, comics, classical music
T4	novel, newspaper, film, comics
T5	novel, newspaper, film, classical music

Say we put minimum support : 0.4, i.e. at least **2 of 5 transactions**.

➜ **Candidate rejected**

- **Join** frequent pairs {film, classical music} and {film, novel} to form {film, classical music, novel}.
- **Prune** : {classical music, novel} occurs only in T5 : $s = 1/5 < 0.4$.
- Reject the triple **without counting it**.

Apriori : A tiny example

ID	Items
T1	novel, newspaper, music
T2	novel, film, comics, contemporary music
T3	newspaper, film, comics, classical music
T4	novel, newspaper, film, comics
T5	novel, newspaper, film, classical music

Say we put minimum support : 0.4, i.e. at least **2 of 5 transactions**.

➜ **Candidate rejected**

- **Join** frequent pairs {film, classical music} and {film, novel} to form {film, classical music, novel}.
- **Prune** : {classical music, novel} occurs only in T5 : $s = 1/5 < 0.4$.
- Reject the triple **without counting it**.

➜ **Candidate retained**

- **Join** frequent pairs {film, newspaper} and {film, comics} to form {film, newspaper, comics}.
- **Prune ?** Its three pairs have supports $3/5$, $3/5$, and $2/5$. All are frequent.
- **Count** : the triple occurs in T3 and T4 : $s = 2/5 = 0.4 \Rightarrow$ keep.

A Tree-based approach : FP-Growth Han et al. 2000

Alternative (& advanced) Algorithms

FP-Growth uses a compact data structure called an FP-Tree (Frequent Pattern Tree)

➜ **Phase 1 : Build the FP-Tree**

- Pass 1 : Count items, retain frequent ones, and order them from most to least frequent.
- Pass 2 : Insert each transaction's frequent items in this order. Merge common prefixes and maintain counts.

A Tree-based approach : FP-Growth Han et al. 2000

Alternative (& advanced) Algorithms

FP-Growth uses a compact data structure called an FP-Tree (Frequent Pattern Tree)

➜ **Phase 1 : Build the FP-Tree**

- Pass 1 : Count items, retain frequent ones, and order them from most to least frequent.
- Pass 2 : Insert each transaction's frequent items in this order. Merge common prefixes and maintain counts.

➜ **Phase 2 : Mine the Tree (Pattern Growth)**

- Extract frequent itemsets from the FP-Tree.
- Pattern growth : Starting with the least frequent items, grow patterns recursively using conditional pattern bases (prefix paths and their counts).
- Divide-and-conquer approach avoids generating candidates that don't exist in the data (Apriori-style candidate generation).

Other approaches

Alternative (& advanced) Algorithms

- The Park-Chen-Yu (PCY) algorithm (Park, Chen and Yu, 1995)
  - Use a hash table in Pass 1 to pre-filter pairs and reduce memory needs in Pass 2
  - Good when when memory for candidate pairs is the bottleneck.
- Savasere-Omiecinski-Navathe (SON) Algorithm (Savasere, Omiecinski and Navathe, 1995)
  - Use data chunks to find candidate itemsets in parallel, then verify globally in Pass 2
  - Good for large datasets that don't fit in memory ; distributed computing.
- Toivonen's Algorithm (Toivonen, 1996)
  - Find candidates from a random sample, then verify against the full data
  - Recover missing frequent itemsets if necessary
- FP-max (Grahne and Zhu, 2003)
  - A variant of FP-Growth, specifically to mine only maximal frequent itemsets
  - Good for concise patters (but not exhaustive)

Rules and rule evaluation

From Frequent Itemsets to Rules

- Once a frequent itemset is found (e.g., $L = \{\text{newspaper, film, comics}\}$), we generate rules by partitioning it.
- For each non-empty subset $A \subset L$, we form the candidate rule $A \Rightarrow (L \setminus A)$.
- **Example Candidates from {newspaper, film, comics} :**
  - $\{\text{newspaper, film}\} \Rightarrow \{\text{comics}\}$
  - $\{\text{newspaper, comics}\} \Rightarrow \{\text{film}\}$
  - $\{\text{newspaper}\} \Rightarrow \{\text{film, comics}\}$
  - ... etc

From Frequent Itemsets to Rules

- Once a frequent itemset is found (e.g., $L = \{\text{newspaper, film, comics}\}$), we generate rules by partitioning it.
- For each non-empty subset $A \subset L$, we form the candidate rule $A \Rightarrow (L \setminus A)$.
- **Example Candidates from {newspaper, film, comics} :**
  - $\{\text{newspaper, film}\} \Rightarrow \{\text{comics}\}$
  - $\{\text{newspaper, comics}\} \Rightarrow \{\text{film}\}$
  - $\{\text{newspaper}\} \Rightarrow \{\text{film, comics}\}$
  - ... etc
- We then calculate the confidence for each candidate rule using the supports we already found.

$$c(A \Rightarrow L \setminus A) = \frac{s(L)}{s(A)}$$

- Keep only the rules that meet or exceed the *minconf* threshold.

Is Confidence Enough ?

A Misleading Rule

Consider the rule $\{\text{Tea}\} \Rightarrow \{\text{Coffee}\}$.

➜ The rule might have high confidence (e.g., 75%), because coffee is an extremely popular item that many people buy, regardless of whether they buy tea.

Is Confidence Enough ?

A Misleading Rule

Consider the rule $\{\text{Tea}\} \Rightarrow \{\text{Coffee}\}$.

- The rule might have high confidence (e.g., 75%), because coffee is an extremely popular item that many people buy, regardless of whether they buy tea.
- Let's say 80% of all customers buy coffee anyway.
- In this case, knowing a customer bought tea actually *decreases* the probability they will buy coffee (from 80% down to 75%).

Is Confidence Enough ?

A Misleading Rule

Consider the rule $\{\text{Tea}\} \Rightarrow \{\text{Coffee}\}$.

- The rule might have high confidence (e.g., 75%), because coffee is an extremely popular item that many people buy, regardless of whether they buy tea.
- Let's say 80% of all customers buy coffee anyway.
- In this case, knowing a customer bought tea actually *decreases* the probability they will buy coffee (from 80% down to 75%).
- The rule is not "interesting", even with high confidence. We need a better measure of "interestingness".

When is a Rule Truly Interesting ?

Statistical Relationship Between Items

A rule $X \Rightarrow Y$ is interesting only if the items are **positively correlated**.

- The condition for statistical independence is :

$$P(Y|X) = P(Y), \text{ which is the same as } c(X \Rightarrow Y) = s(Y)$$

- Equivalent to : $P(X \cup Y) = P(X) \times P(Y)$; probability that both X and Y would appear in a transaction if their occurrences were completely independent
- This means that knowing X occurred gives us no new information about Y.

Correlation

We can compare the actual co-occurrence probability with the expected probability under independence :

- If $P(X \cup Y) > P(X) \times P(Y) \implies$ X and Y are **positively correlated**.
- If $P(X \cup Y) < P(X) \times P(Y) \implies$ X and Y are **negatively correlated**.

Our goal is to find rules that represent positive correlations.

Beyond Support-Confidence : Lift

Lift

Lift measures how much more likely Y is to be purchased when X is purchased, while controlling for the popularity of Y.

$$\text{Lift}(X \Rightarrow Y) = \frac{\text{Confidence}(X \Rightarrow Y)}{\text{Support}(Y)} = \frac{s(X \cup Y)}{s(X)s(Y)}$$

Interpreting Lift

- **Lift = 1 :** X and Y are independent. There is no real relationship.
- **Lift > 1 :** The presence of X increases the likelihood of Y (positive correlation). This is an interesting rule !
- **Lift < 1 :** The presence of X decreases the likelihood of Y (negative correlation).

Beyond Support-Confidence

- Leverage : Difference between observed and expected co-occurrence.
- Conviction : Measures the dependence of consequent on the antecedent (higher means stronger rule)
- Zhang's Metric : Quantitative measure of association or dissociation
- Jaccard : Similarity measure of overlap between antecedent and consequent.
- Certainty : Increase of the probability of the consequent given the antecedent.
- Kulczynski : Symmetric measure averaging conditional probabilities both ways.

In conclusion

Recap

- **Association Rules** uncover valuable co-occurrence patterns in large datasets, with applications from retail to medicine.
- The core metrics are **Support** (popularity) and **Confidence** (predictive power).
- The **A Priori algorithm** efficiently finds frequent itemsets by intelligently pruning the search space.
- Advanced algorithms offer significant performance and memory improvements for large-scale problems.
- Measures like **Lift** (and other) are crucial for evaluating whether discovered patterns are genuinely interesting and not just statistical artifacts.

Further reading

➜ Seminal Papers

- Agrawal, R., & Srikant, R. (1994). *Fast algorithms for mining association rules.* The foundational paper on the Apriori algorithm.
- Han, J., Pei, J., & Yin, Y. (2000). *Mining frequent patterns without candidate generation.* Introduced the highly influential FP-Growth algorithm.

➜ Textbooks

- Han, J., Kamber, M., & Pei, J. (2011).
  Data Mining : Concepts and Techniques. (Chapter on Frequent Pattern Mining).
- Tan, P. N., Steinbach, M., Karpatne, A. & Kumar, Karpatne(2006).
  Introduction to data mining. (Chapter on Association Analysis)
  `https://www-users.cse.umn.edu/~kumar001/dmbook/index.php`

Bridging the Gap : FCA and Association Rules

One last thing...

Formal Concept Analysis (FCA)

Finds **Attribute Implications** :

$$Z_1 \implies Z_2$$

This means : "Every object that has all attributes in Z_1 *also* has all attributes in Z_2."

- **Deterministic** rule.
- Holds for **all** relevant objects in the data/context

Association Rule Mining

Finds **Association Rules** :

$$X \Rightarrow Y$$

This means : "Transactions containing items in X are *likely* to also contain items in Y."

- **Probabilistic** rule.
- Measured by support and confidence (and Lift etc.)

An attribute implication from FCA is a special case of an association rule. An implication $Z_1 \implies Z_2$ is an association rule $Z_1 \Rightarrow Z_2$ with **Confidence = 100%**

Can you show it ?

A note on the lab exercises

- TAs
  - Capucine Brillet
  - Mohamed Islem Kara Bernou
- Requirements
  - Python 3.10+
  - Jupyter Notebook or JupyterLab (for running the notebooks)
- Two sets of lab exercises
  - Formal Concept Analysis : French Recipes
  - Association Rules : Movie Recommendations
- Get the labs :
  - Moodle (.zip)
  - Git repository : link on moodle or :
    `https://gitlab.telecom-paris.fr/simidjievskin/logicai20262027`

Questions

Appendix

NextClosure

Formal Context & Setup

The Context (X, Y, I) :

I	y_1	y_2	y_3	y_4
x_1	×	×	×	×
x_2	×		×	×
x_3		×	×	×
x_4		×	×	×
x_5	×			

Definitions :

- Object set $X = \{x_1, ..., x_5\}$
- Attribute set $Y = \{y_1, ..., y_4\}$
- Lectical Order : $y_1 < y_2 < y_3 < y_4$

Object Intents ($x^{\uparrow}$) :

- $x_1^{\uparrow} = \{y_1, y_2, y_3, y_4\}$
- $x_2^{\uparrow} = \{y_1, y_3, y_4\}$
- $x_3^{\uparrow} = \{y_2, y_3, y_4\}$
- $x_4^{\uparrow} = \{y_2, y_3, y_4\}$
- $x_5^{\uparrow} = \{y_1\}$

Attribute Extents ($y^{\downarrow}$) :

- $y_1^{\downarrow} = \{x_1, x_2, x_5\}$
- $y_2^{\downarrow} = \{x_1, x_3, x_4\}$
- $y_3^{\downarrow} = \{x_1, x_2, x_3, x_4\}$
- $y_4^{\downarrow} = \{x_1, x_2, x_3, x_4\}$

Iteration 0 : Finding Concept C_1

- Start with the empty set of attributes, $\emptyset$.
- Since we are not requiring any attribute, **all objects match** :

$$\emptyset^{\downarrow} = \{x_1, x_2, x_3, x_4, x_5\}.$$

- Now ask which attributes are shared by all five objects :

$$\emptyset^{\downarrow\uparrow} = \emptyset.$$

- No attribute is common to all objects, so $\emptyset$ is already a closed set and therefore the first intent.

Concept 1

$$C_1 = (\{x_1, x_2, x_3, x_4, x_5\}, \emptyset)$$

Iteration 1 : Finding Concept C_2

- **Current intent :** $\emptyset$. To find the next one, try attributes from right to left, starting with y_4.
- First try $\{y_4\}$. The objects having y_4 are :

$$\{y_4\}^{\downarrow} = \{x_1, x_2, x_3, x_4\}.$$

These objects all share both y_3 and y_4 :

$$\{y_4\}^{\downarrow\uparrow} = \{y_3, y_4\}.$$

- This means that trying y_4 also forces the earlier attribute y_3, so $\{y_4\}$ does not give the next intent.
- Try y_3 instead :

$$\{y_3\}^{\downarrow} = \{x_1, x_2, x_3, x_4\}, \qquad \{y_3\}^{\downarrow\uparrow} = \{y_3, y_4\}.$$

- No attribute before y_3 is added, so $\{y_3, y_4\}$ is the next intent.

Concept 2

$$C_2 = (\{x_1, x_2, x_3, x_4\}, \{y_3, y_4\})$$

Iteration 2 : Finding Concept C_3

- **Current intent :** $\{y_3, y_4\}$. Since y_4 and y_3 are already present, try y_2 next.
- Start with the candidate $\{y_2\}$. The objects having y_2 are :

$$\{y_2\}^{\downarrow} = \{x_1, x_3, x_4\}.$$

- These objects also all have y_3 and y_4, so closing $\{y_2\}$ gives :

$$\{y_2\}^{\downarrow\uparrow} = \{y_2, y_3, y_4\}.$$

- Therefore $\{y_2, y_3, y_4\}$ is the next intent.

Concept 3

$$C_3 = (\{x_1, x_3, x_4\}, \{y_2, y_3, y_4\})$$

Iteration 3 : Finding Concept C_4

- **Current intent :** $\{y_2, y_3, y_4\}$. The attributes y_4, y_3 and y_2 are already present, so the next attribute to try is y_1.
- Since there is no attribute before y_1, we start from $\{y_1\}$.
- The objects having y_1 are :

$$\{y_1\}^{\downarrow} = \{x_1, x_2, x_5\}.$$

- The only attribute shared by these three objects is y_1 :

$$\{y_1\}^{\downarrow\uparrow} = \{y_1\}.$$

- Therefore $\{y_1\}$ is already closed and becomes the next intent.

Concept 4

$$C_4 = (\{x_1, x_2, x_5\}, \{y_1\})$$

Iteration 4 : Finding Concept C_5

- **Current intent :** $\{y_1\}$. Start again from the right and first try adding y_4.
- The objects having both y_1 and y_4 are :

$$\{y_1, y_4\}^{\downarrow} = \{x_1, x_2\}.$$

- These two objects also both have y_3 :

$$\{y_1, y_4\}^{\downarrow\uparrow} = \{y_1, y_3, y_4\}.$$

Trying y_4 therefore also introduces the earlier attribute y_3, so this is not the next intent.

- Try y_3 instead :

$$\{y_1, y_3\}^{\downarrow} = \{x_1, x_2\}, \qquad \{y_1, y_3\}^{\downarrow\uparrow} = \{y_1, y_3, y_4\}.$$

- No new attribute before y_3 is introduced, so $\{y_1, y_3, y_4\}$ is the next intent.

Concept 5

$$C_5 = (\{x_1, x_2\}, \{y_1, y_3, y_4\})$$

Iteration 5 : Finding Concept C_6

- **Current intent :** $\{y_1, y_3, y_4\}$. The attributes y_4 and y_3 are already present, so the next attribute to try is y_2.
- Keep y_1, because it comes before y_2, and add y_2. We therefore close $\{y_1, y_2\}$.
- Only one object has both attributes :

$$\{y_1, y_2\}^{\downarrow} = \{x_1\}.$$

- Object x_1 has all four attributes :

$$\{y_1, y_2\}^{\downarrow\uparrow} = \{y_1, y_2, y_3, y_4\}.$$

- The attribute before y_2, namely y_1, has not changed, so the full attribute set is the next intent.

Concept 6

$$C_6 = (\{x_1\}, \{y_1, y_2, y_3, y_4\})$$

Termination

- The current intent is now the full attribute set :

$$B = \{y_1, y_2, y_3, y_4\} = Y.$$

- Every attribute is already present, so there is no further attribute that can produce a new intent.
- NextClosure has generated all six intents, and the algorithm terminates.

All Formal Concepts

The 6 concepts order in the hierarchy :

1. $C_1 = (\{x_1, x_2, x_3, x_4, x_5\}, \emptyset)$ - **most general**, includes all objects
2. — 2nd Level (sub-concepts of C_1)
3. $C_2 = (\{x_1, x_2, x_3, x_4\}, \{y_3, y_4\})$
4. $C_4 = (\{x_1, x_2, x_5\}, \{y_1\})$
5. — 3rd Level (both sub-concepts of C_2, & C_5 of C_4)
6. $C_3 = (\{x_1, x_3, x_4\}, \{y_2, y_3, y_4\})$
7. $C_5 = (\{x_1, x_2\}, \{y_1, y_3, y_4\})$
8. —
9. $C_6 = (\{x_1\}, \{y_1, y_2, y_3, y_4\})$ - **most specific**

Association Rules algorithms

Association Rules algorithms

	# of passes	Idea	When to Use
Apriori	k (for size k)	Prune candidates level-by-level using the anti-monotone property of support.	Works for smaller datasets.
PCY	2 (for pairs)	Use a hash table in Pass 1 to prefilter pairs and reduce memory needs in Pass 2	When memory for candidate pairs is the bottleneck.
SON	2	Use data chunks to find candidate itemsets in parallel, then verify globally in Pass 2	Very large datasets that don't fit in memory ; distributed computing.
Toivonen	1, or 2 if needed	Verify sample candidates and the negative border ; recover missing frequent itemsets if needed.	Exact results with sampling ; extra work if verification finds omissions.
FP-Growth	2	Count, build FP-tree, then mine ; avoids Apriori-style candidate generation.	When Apriori candidate generation is costly and the tree fits available memory.